P.PORTO

Sistemas Distribuídos

LICENCIATURA EM ENGENHARIA INFORMÁTICA · LICENCIATURA EM SEGURANÇA INFORMÁTICA EM REDES DE COMPUTADORES

T03 · 12 de outubro de 2026

Retomando: T02, PL01 e PL02

- **T02:** seis objetivos de desenho (transparência, abertura, escalabilidade…)
- **A pergunta de hoje:** como se organiza um sistema para os atingir? — arquitetura
- **PL01:** servidor Echo — um cliente de cada vez
- **PL02:** um servidor com uma thread por cliente; chat que difunde a todos
- **Hoje:** estilos arquiteturais → centralizadas → descentralizadas → híbridas e middleware

O que é um estilo arquitetural

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Architectural styles; System architectures (organização do capítulo)

- **Componente —** unidade substituível, com interfaces bem definidas
- **Conector —** mecanismo que medeia a comunicação, coordenação ou cooperação entre componentes (chamada remota, mensagens, fluxos de dados)
- **Estilo (architectural style) —** componentes, modo como se ligam, dados trocados e configuração conjunta
- **Arquitetura de software —** como os componentes se organizam e interagem (os estilos)
- **Arquitetura de sistema —** onde os componentes são colocados em máquinas reais

Estilo em camadas (layered)

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Layered architecture

- **Componentes organizados em camadas:** cada uma usa os serviços da camada de baixo e oferece serviços à de cima
- **Exemplo:** pilha de protocolos de comunicação — cada camada com o seu protocolo, serviço e interface
- **O livro ilustra o estilo com um par servidor/cliente por sockets (accept, connect, send, recv):** o mesmo esqueleto do Echo da PL01
- **As próprias aplicações também se organizam em camadas** (próximos blocos)

Estilo baseado em objetos

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Object-based style

- **Componentes são objetos,** ligados por chamadas a procedimentos (métodos)
- **Os objetos podem estar em máquinas diferentes:** a chamada executa através da rede
- **Encapsulamento:** o objeto esconde a implementação e oferece métodos sobre os seus dados
- No livro, agrupado com o REST sob «arquiteturas orientadas a serviços» (service-oriented)
- **Como a chamada remota funciona por dentro:** matéria de aulas futuras

Estilo baseado em recursos (REST)

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — RESTful architectures

- **Essência:** o SD como coleção de recursos geridos por componentes, que aplicações (remotas) criam, leem, modificam e apagam
- **Quatro regras:** um só esquema de nomes; todos os serviços com a mesma interface; mensagens autodescritivas; após cada operação o serviço esquece o chamador (sem estado)
- **Operações básicas:** POST (criar), GET (ler o estado), PUT (modificar), DELETE (apagar)
- **Exemplo do livro:** Amazon S3 — objetos em «buckets», identificados por URI e acedidos por pedidos HTTP

Publicação-subscrição (publish-subscribe)

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Publish-subscribe architectures (coordenação)

- **Dois eixos:** referencial — o emissor nomeia o destinatário? · temporal — têm de estar ativos ao mesmo tempo?
- **Direto —** acoplado nos dois eixos
- **Baseado em eventos —** desacoplado na referência, acoplado no tempo
- **Caixa de correio (mailbox) —** acoplado na referência, desacoplado no tempo
- **Espaço de dados partilhado (shared data space) —** desacoplado nos dois eixos

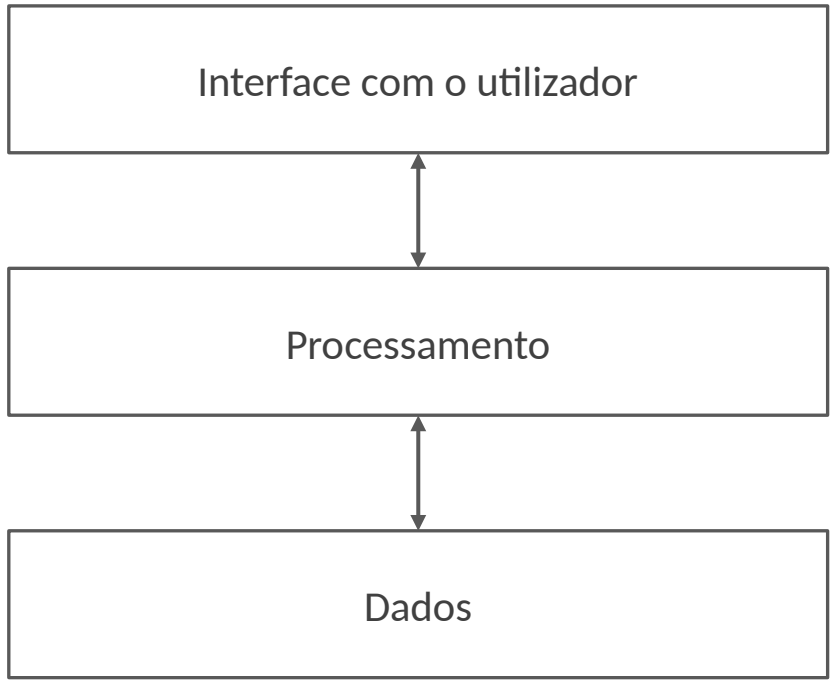
E o chat da PL02? Que estilo é?

- **Clientes ligam-se por TCP a um servidor,** que reenvia cada linha a todos os clientes ligados
- **Pergunta 1:** em que eixos estão emissor e destinatários acoplados — referência e tempo?
- **Pergunta 2:** que estilo do slide anterior se aproxima mais?
- **Pistas:** o emissor escolhe quem recebe? E um cliente desligado recebe depois a mensagem?

Cliente-servidor e camadas aplicacionais

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Centralized system architectures; Application layering

- **Cliente-servidor:** servidores oferecem serviços, clientes usam-nos; podem estar em máquinas diferentes; modelo pedido/resposta (request/reply)
- **Interface —** contacto com utilizadores ou outras aplicações
- **Processamento —** as funções da aplicação, sem dados específicos
- **Dados —** os dados que a aplicação manipula



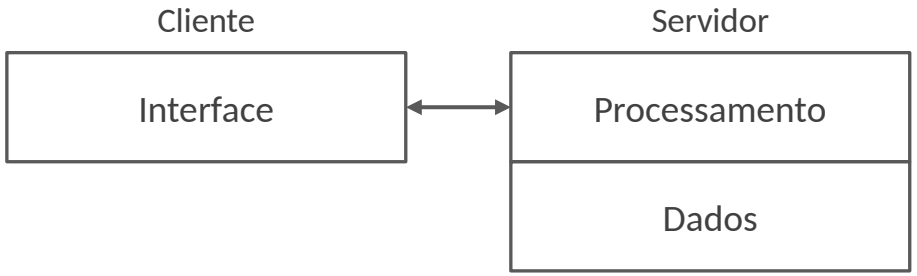
Camadas lógicas de uma aplicação

Duas camadas: cliente leve ou pesado

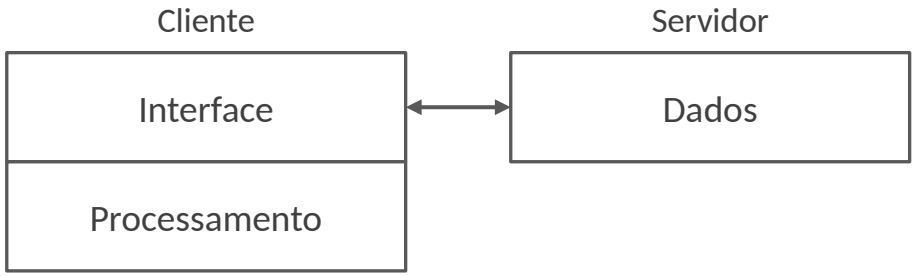
Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Multitiered architectures (two-tiered)

- **Duas camadas (two-tiered):** clientes e um único servidor — varia o ponto onde se corta entre cliente e servidor
- **Leve:** o cliente só trata a interface · **Pesado:** o cliente executa também o processamento

Cliente leve (thin client)



Cliente pesado (fat client)



Cada caixa é uma camada lógica; o que muda é a máquina onde corre.

Multicamada (multitiered) e a Web

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Multitiered architectures; Example: The Web

- **Três camadas (three-tiered):** cada camada lógica numa máquina diferente — distribuição vertical (vertical distribution)
- **Web:** de ficheiros HTML servidos tal e qual a páginas compostas por um programa (CGI) sobre uma base de dados



servidor e cliente ao mesmo tempo

Exercício: quantas camadas?

Para cada sistema: quantas camadas físicas? Cliente leve ou pesado?

1. Chat da PL02: clientes de consola e um servidor Java que guarda em memória a lista de clientes e difunde as mensagens
2. Aplicação de reservas instalada no PC de cada utilizador: toda a lógica corre no PC; só a base de dados está num servidor partilhado
3. Loja online: o browser fala com um servidor aplicacional, que consulta uma base de dados noutra máquina

Arquiteturas descentralizadas e P2P

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Symmetrically distributed system architectures

- **Vertical:** camadas lógicas em máquinas diferentes (slide 11)
- **Horizontal:** um cliente ou servidor dividido em partes logicamente equivalentes, cada uma com a sua parcela dos dados
- **Peer-to-peer (P2P):** todos os processos são iguais — cada um é cliente e servidor ao mesmo tempo
- **Rede de sobreposição (overlay network):** os pares ligam-se logicamente entre si
- **Duas famílias:** estruturada (índice por chave) e não estruturada (vizinhos ad hoc)

P2P estruturado: tabela de hash distribuída

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Structured P2P; Example: Chord (só a ideia)

- **Índice sem semântica:** cada item de dados tem uma chave = hash(valor do item)
- **O sistema é responsável por guardar pares (chave, valor);** pesquisar a chave k é encaminhar o pedido até ao nó responsável
- **Chord:** nós num anel, cada um com um identificador de m bits; o item de chave k fica no nó de menor identificador ≥ k (o sucessor de k)
- **Atalhos (shortcuts) entre nós** aceleram a pesquisa

P2P não estruturado e super-pares

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Unstructured P2P; Super-peer networks

- **Não estruturado:** cada nó mantém uma lista ad hoc de vizinhos; a rede lembra um grafo aleatório
- **Pesquisa por inundação (flooding):** o nó pergunta a todos os vizinhos, que repetem; ignoram-se pedidos já vistos; limite de saltos (TTL)
- **Alternativa:** passeio aleatório (random walk) — pergunta a um vizinho de cada vez
- **Super-pares (super-peers):** quebram a simetria — mantêm índices e fazem de intermediários (brokers); rede hierárquica

BitTorrent: um exemplo cruzado

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Collaboration: The BitTorrent case

- **Procurar o ficheiro F** num diretório global, que devolve um ficheiro torrent
- **O torrent referencia um tracker:** servidor que regista os nós ativos que têm pedaços (chunks) de F
- **Um par junta-se ao swarm,** obtém um pedaço e troca uma cópia dele por outro com outro par
- **Pergunta:** que partes são cliente-servidor e que parte é P2P?

Híbridas: nuvem e periferia (edge)

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Hybrid system architectures

- **Nuvem (cloud computing), quatro camadas:** hardware; infraestrutura (virtualização); plataforma (ex.: S3); aplicação
- **Periferia (edge):** servidores na fronteira entre redes empresariais e a Internet, mais perto do utilizador
- **Argumentos habituais:** latência e largura de banda, fiabilidade, segurança e privacidade — o autor adverte que nem todos resistem
- **Também no capítulo:** blockchains — só reconhecimento

Middleware: abertura e transparência

Tanenbaum & Van Steen, 4.ª ed., cap. 2 — Middleware and distributed systems

- **Middleware:** «o sistema operativo dos sistemas distribuídos» — componentes e funções comuns que as aplicações não têm de implementar uma a uma
- **Ligação à T02:** é onde se constroem a transparência de distribuição e a abertura (interfaces padronizadas)
- **Adaptadores (wrappers):** dão a componentes antigos (legacy) uma interface aceitável
- **Com um mediador (broker):** 2N adaptadores em vez de N×(N−1)

Resumo

- **Estilo arquitetural:** componentes, conectores, dados trocados e configuração
- **Estilos:** camadas, objetos, recursos (REST), publicação-subscrição
- **Centralizadas:** cliente-servidor; duas camadas (leve/pesado) e multicamada
- **Descentralizadas:** P2P estruturado e não estruturado; super-pares
- **Híbridas e middleware:** nuvem e periferia; abertura e transparência